

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

ОТЧЕТ
О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ
«ДИНАМИКА СИСТЕМЫ»
ПО ДИСЦИПЛИНЕ «ТЕОРЕТИЧЕСКАЯ МЕХАНИКА И
ОСНОВЫ КОМПЬЮТЕРНОГО МОДЕЛИРОВАНИЯ»
ВАРИАНТ ЗАДАНИЯ № 9

Выполнил(а) студент группы М80-209Б-23

Калинин Иван Николаевич _____
подпись, дата

Проверил и принял

Ст. преп. каф. 802 Волков Е.В. _____
подпись, дата

с оценкой _____

Москва, 2024

Задание: Построить анимацию движения системы с помощью Python.
Построить графики зависимостей $\Phi(t)$, $\Psi(t)$, $R_x(t)$, $R_y(t)$.

Листинг:

```
import numpy as np
import matplotlib.pyplot as plt
import math
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation

# начальные значения
Steps = 500
t = np.linspace(0, 20, Steps)
m1 = 2
m2 = 3
m3 = 5
l = 4
r = 1
c = 1
g = 9.8
WheelR = r

psi0 = math.pi / 8
phi0 = math.pi / 8
dpsi0 = 1
dphi0 = 1

y0 = [phi0, psi0, dphi0, dpsi0]

Nv = 2
R1 = 0.1
R2 = 0.3

# система дифференциальных уравнений для вычисления законов движения от t
def odesys(y, t, m1, m2, m3, l, r, c):
    dy = np.zeros(4)
    dy[0] = y[2]
    dy[1] = y[3]
    a11 = 1 * (m1 / 3 + m2 + m3)
    a12 = m3 * r * np.cos(y[0] + y[1])
    a21 = m3 * l * np.cos(y[0] + y[1])
    a22 = (m2 / 2 + m3) * r
    b1 = -(m1 / 2 + m2 + m3) * g * np.sin(y[0]) - c / l * (y[1] + y[0]) + m3
    b2 = m3 * g * np.sin(y[1]) - c / r * (y[0] + y[1]) + m3 * l * (y[2] ** 2)
    dy[2] = (b1 * a22 - b2 * a12) / (a11 * a22 - a12 * a21)
    dy[3] = (b2 * a11 - b1 * a21) / (a11 * a22 - a12 * a21)
    return dy

Y = odeint(odesys, y0, t, (m1, m2, m3, l, r, c))
phi = Y[:, 0]
psi = Y[:, 1]
dphi = Y[:, 2]
dpsi = Y[:, 3]
ddphi = [odesys(y0, t, m1, m2, m3, l, r, c)[2] for y0, t in zip(Y, t)]
ddpsi = [odesys(y0, t, m1, m2, m3, l, r, c)[3] for y0, t in zip(Y, t)]

Rx = -(m1 / 2 + m2 + m3) * l * (ddphi * np.sin(phi) + dphi ** 2 *
np.cos(phi)) + m3 * r * (
```

```

        ddppsi * np.sin(psi) - psi ** 2 * np.sin(psi))
Ry = (m1 / 2 + m2 + m3) * l * (ddphi * np.sin(phi) + dphi ** 2 * np.cos(phi))
+ m3 * r * (
        ddppsi * np.sin(psi) - psi ** 2 * np.sin(psi))

fig = plt.figure(figsize=[10, 5])
ax = fig.add_subplot(1, 2, 1)
ax.axis('equal')
ax.set(xlim=[-7, 7], ylim=[-7, 3])
X_Ground = [-1, 1]
Y_Ground = [0, 0]
ax.plot(X_Ground, Y_Ground, color='black', linewidth=4)

X_A = l * np.sin(phi)
Y_A = -l * np.cos(phi)

ax2 = fig.add_subplot(4, 2, 2)
ax2.plot(phi)
plt.title('Phi(t)')
plt.xlabel('t')
plt.ylabel('rad')
ax2 = fig.add_subplot(4, 2, 4)
ax2.plot(psi)
plt.title('Psi(t)')
plt.xlabel('t')
plt.ylabel('rad')
ax2 = fig.add_subplot(4, 2, 6)
ax2.plot(Rx)
plt.title('Rx(t)')
plt.xlabel('t')
plt.ylabel('N')
ax2 = fig.add_subplot(4, 2, 8)
ax2.plot(Ry)
plt.title('Ry(t)')
plt.xlabel('t')
plt.ylabel('N')
plt.subplots_adjust(wspace=0.3, hspace=1.3)

tetta = np.linspace(0, 2 * math.pi, Steps)
X_Wheel = WheelR * np.sin(tetta)
Y_Wheel = WheelR * np.cos(tetta)
Drawed_Wheel = ax.plot(X_A[0] + X_Wheel, Y_A[0] + Y_Wheel)[0]
Point_O = ax.plot(0, 0, marker='o')[0]
Point_A = ax.plot(X_A[0], Y_A[0], marker='o')[0]
Line_AO = ax.plot([X_A[0], 0], [Y_A[0], 0])[0]
A = np.sin(psi[0]) * WheelR
X_B = X_A + r * np.sin(psi)
Y_B = Y_A + r * np.cos(psi)
Point_B = ax.plot(X_B[0], Y_B[0], marker='o')[0]
Line_AB = ax.plot([X_A[0], X_B[0]], [Y_A[0], Y_B[0]])[0]

gretta = np.linspace(0, Nv * 2 * math.pi - psi[0], Steps)
X_SpiralSpr = (R1 + gretta * (R2 - R1) / gretta[-1]) * np.sin(gretta)
Y_SpiralSpr = (R1 + gretta * (R2 - R1) / gretta[-1]) * np.cos(gretta)
Drawed_SpiralSpring = ax.plot(X_SpiralSpr + X_A[0], Y_SpiralSpr + Y_A[0])[0]

def anima(i):
    Line_AO.set_data([X_A[i], 0], [Y_A[i], 0])
    Point_A.set_data(X_A[i], Y_A[i])
    Drawed_Wheel.set_data(X_A[i] + X_Wheel, Y_A[i] + Y_Wheel)
    Point_B.set_data(X_B[i], Y_B[i])
    Line_AB.set_data([X_A[i], X_B[i]], [Y_A[i], Y_B[i]])
    gretta = np.linspace(0, Nv * 2 * math.pi - psi[i], Steps)

```

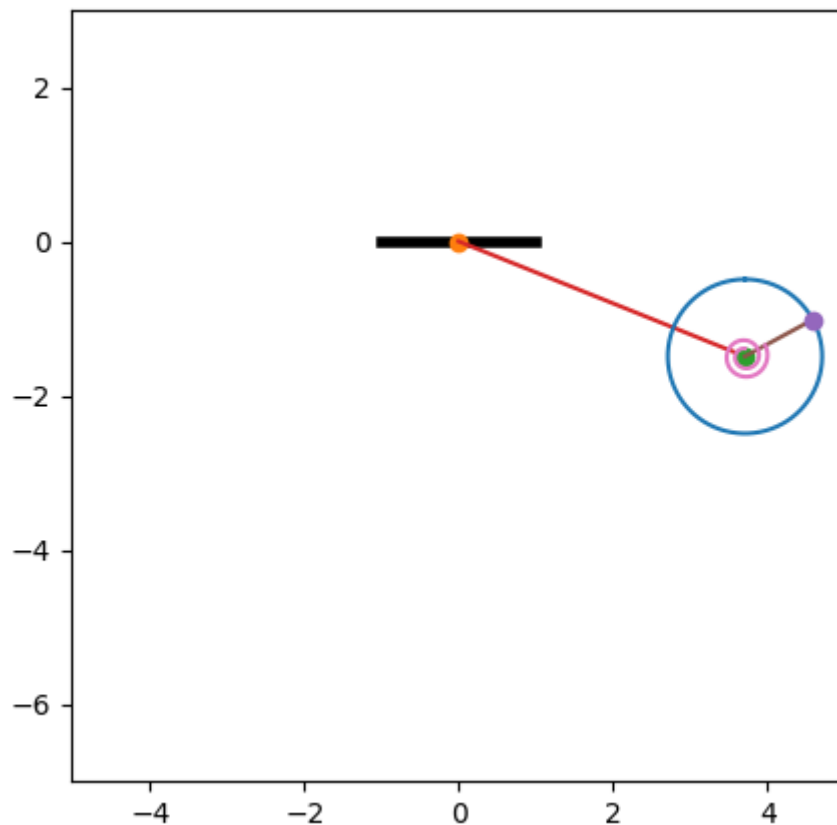
```

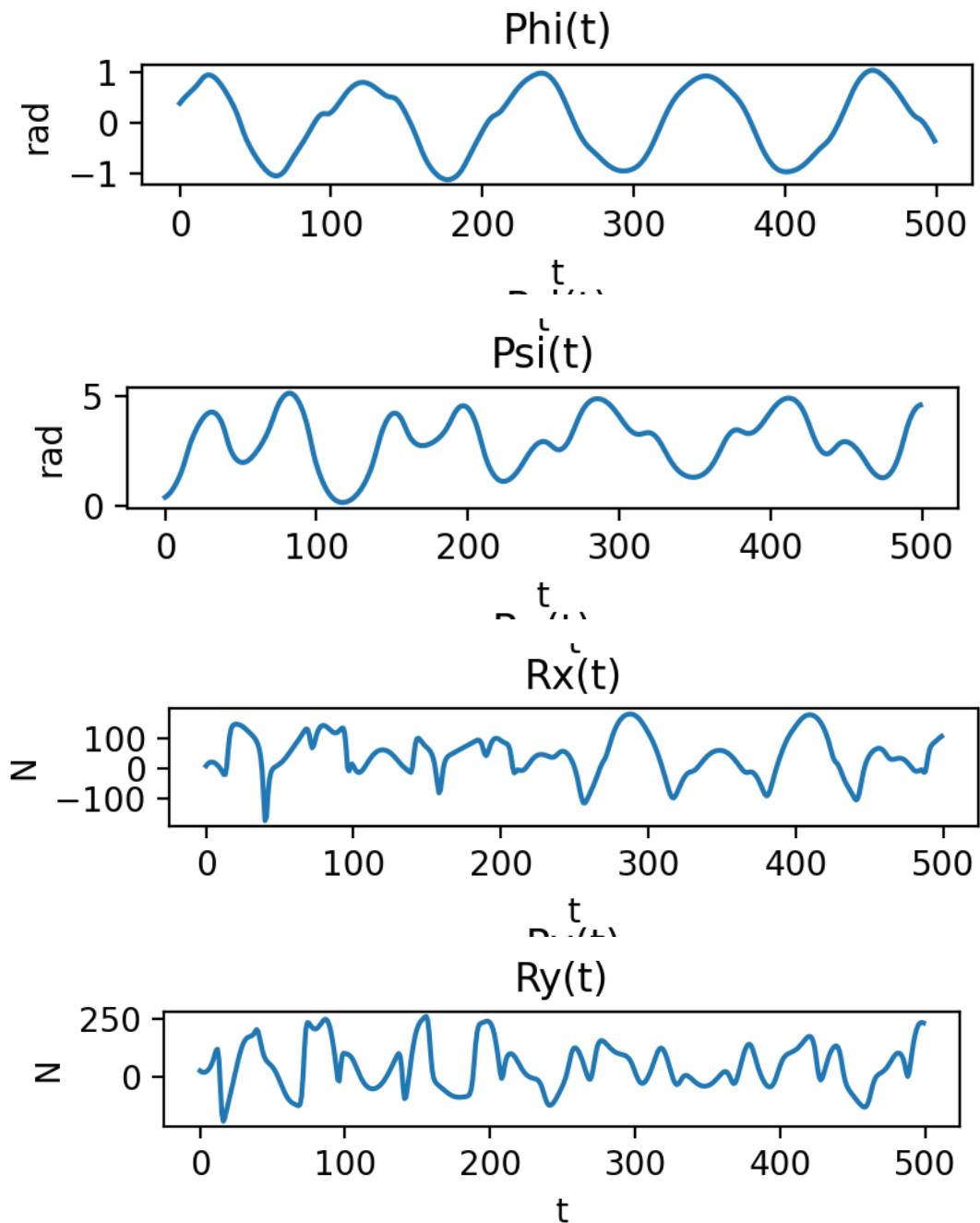
X_SpiralSpr = -(R1 + gretta * (R2 - R1) / gretta[-1]) * np.sin(gretta)
Y_SpiralSpr = (R1 + gretta * (R2 - R1) / gretta[-1]) * np.cos(gretta)
Drawed_SpiralSpring.set_data(X_SpiralSpr + X_A[i], Y_SpiralSpr + Y_A[i])
return [Line_AO, Point_A, Drawed_Wheel, Line_AB, Point_B,
        Drawed_SpiralSpring]

anim = FuncAnimation(fig, anima, frames=Steps, interval=100, blit=True)
plt.show()

```

Результаты:





Вывод: Выполнив лабораторную работу я получил опыт работы с графическими библиотеками языка Python. Научился строить графики сложных функций. Приобрел практические навыки в области анимации системы.